

PROJECT G · FIELD HANDBOOK

On-Device Real-Time Segmentation Pipeline

A complete beginner's playbook — concepts, a 20-day plan, and what to do each day

SCANNER-01 · NVIDIA Jetson Orin · 4 weeks / 20 working days · assumes no prior experience

The whole project in one paragraph

You're building a system that looks at the SCANNER-01's camera feed and labels what it sees — **in real time, on the device itself**. The old plan (train a model from scratch on hand-labelled images) was dropped for being too slow. The new plan chains **foundation models**: an open-vocabulary **YOLO** detects and names objects, **NanoSAM** turns each detection into a precise mask, and those masks get lifted onto the LiDAR map — all compiled with **TensorRT** and run in reduced precision so it keeps up with the camera.

How to use this handbook

Sections 1–3 explain the system. Read once.

Section 4 is a glossary — flip back whenever a word is unfamiliar.

Section 5 is how to work so you don't get stuck. Read before Day 1.

Section 6 is the 20-day plan: each day has why it matters, what to read, the concepts, what to do (with commands), pitfalls, and a “done when.” This is your daily driver.

Sections 7–8 are the day-one questions for your lead and the full link list.

Contents

Contents	2
1. The pipeline at a glance	3
2. The model stack	3
3. How real-time actually works	3
4. Words you'll keep hearing	5
5. How to work so you don't get stuck	7
The golden rule	7
Your daily loop	7
When you get stuck (the 30-minute rule)	7
6. The 20-day plan	8
7. Day-one questions for your lead	23
8. Key resources	24
Orientation — read first	24
The models	24
Optimisation — getting to real-time	24
Deployment glue	24

1. The pipeline at a glance

The end goal is a **labelled point cloud**: the 3D map the device builds, where every point also carries a meaning. A plain map tells you **where** things are; a labelled one tells you **what** they are. Three things happen, fast, for every frame:

Stage	What happens
1 · Detect & name	Open-vocabulary YOLO finds objects and labels them from text prompts, producing boxes — fast.
2 · Segment	Each box is handed to NanoSAM, which returns a precise pixel-level mask for that object.
3 · Lift to 3D	Masks are projected onto LiDAR points via the camera–LiDAR calibration, so labels live in the 3D map.

Why on-device matters

Field deployments have little or no connectivity, so everything must run on the Orin for immediate insights. That single constraint is why the model choice changed and why all of Week 3 is about speed.

2. The model stack

These are not four competing options — they're pieces of **one pipeline**, plus the techniques that make it fast.

Piece	Role	One-line summary
YOLO 2026	Detect + name	Real-time, open-vocabulary detector; the fast front-end that produces named boxes.
NanoSAM	Segment	Jetson-optimised Segment-Anything variant; turns a box prompt into a precise mask.
MobileSAM	Lineage	The model NanoSAM is distilled from; NanoSAM reuses its mask decoder.
Open-vocab	Paradigm	Labelling by text description, not a fixed class list; lives inside the YOLO front-end.
Distillation	Technique	Training a small fast “student” to copy a big slow “teacher” — why NanoSAM is small and good.

The pattern to internalise

Open-vocab YOLO (detect + name) → NanoSAM (box → mask) → lift masks onto LiDAR points

A fast detector finds and names objects, then a distilled SAM segments only those regions — you never segment the whole frame, which is also a speed trick.

3. How real-time actually works

Real-time doesn't mean instant — it means the pipeline **keeps up with the camera**. Target: **≥10 Hz**, about **100 ms per frame**. You hit that budget by stacking cheap wins, in this order of impact:

Lever	What it does	Typical gain
1. Model choice	Start with a model built to be fast. You never make a slow model real-time. (Why SegFormer was dropped.)	Decisive
2. TensorRT compile	Recompiles the model into a hardware-specific engine for your Orin GPU. Not optional on Jetson.	Multiple ×
3. Precision (FP16/INT8)	Run in lower precision. FP16 is nearly free; INT8 is faster but needs calibration.	~2× (FP16)
4. Resolution & shape	Smaller input = less work. Segment only the boxes YOLO found, not the whole frame.	Situational
5. Measure honestly	Profile latency, frame drops, and thermals. Prove real-time with numbers, not hope.	Validates

The mental model

Pick a fast model → compile with TensorRT → reduce precision → tune resolution & shape → measure → repeat the cheapest remaining lever until you're inside budget.

4. Words you'll keep hearing

Flip back here whenever a term in the day plan is unfamiliar. Plain-English meanings, not textbook definitions.

Term	Plain-English meaning
Inference	Running a trained model to get predictions (as opposed to training it).
JetPack	NVIDIA's software bundle for Jetson: GPU drivers, CUDA, and core libraries.
CUDA	The layer that lets your code run computations on the NVIDIA GPU.
ONNX	A portable model file format that many different tools can read.
TensorRT	NVIDIA's tool that compiles a model into a fast, hardware-specific "engine."
Engine (.engine)	The compiled, optimised model TensorRT produces. Device-specific — rebuild per machine.
FP32 / FP16 / INT8	Number precision: 32-bit (precise), 16-bit (faster), 8-bit (fastest, needs calibration).
Quantization	Converting a model to lower precision (e.g. INT8) to make it faster and smaller.
Calibration	Measuring a model's value ranges on sample data so INT8 stays accurate.
Latency	Time to process one frame, in milliseconds.
Throughput / FPS	Frames processed per second.
Real-time	Throughput keeps up with the camera (target ≥ 10 Hz).
Foundation model	A large pre-trained model usable out-of-the-box, often without training on your data.
Zero-shot	Using a model on new classes with no extra training.
Open-vocabulary	Labelling by text description rather than a fixed, trained class list.
Detection	Finding objects and drawing boxes around them (with names).
Segmentation	Labelling every pixel — finer than detection.
Mask	The per-pixel outline of one object that segmentation produces.
Prompt (SAM)	The point or box you give NanoSAM to tell it what to segment.
Distillation	Training a small fast model to mimic a big slow one (how NanoSAM was made).
mIoU	Main segmentation accuracy metric: average overlap between predicted and true masks.
Point cloud	The set of 3D (x, y, z) points the LiDAR produces.
Extrinsics	The known geometric transform between two sensors (e.g. camera ↔ LiDAR).
ROS 2	The robotics middleware the whole SCANNER-01 system runs on.
Node	One small program within a ROS 2 system.
Topic	A named channel that nodes publish messages to and subscribe from.

Term	Plain-English meaning
tegrastats	Jetson tool showing live CPU / GPU / memory / temperature.
Throttling	The chip slowing itself down when too hot, which lowers your FPS.

5. How to work so you don't get stuck

You've never done this before — that's fine. The people who do well aren't the ones who already know everything; they're the ones with a good loop for learning and unblocking. Here's that loop.

The golden rule

Make it work, then make it fast. Get a correct-but-slow pipeline first (Weeks 1–2). All optimisation is Week 3. Chasing speed before correctness is the single most common way to lose a week.

Your daily loop

- **Read first.** Each day starts with reading because understanding the concept before typing saves far more time than it costs.
- **Do in small steps.** Change one thing, run it, confirm it worked, then move on. Don't write 100 lines and hope.
- **Commit at end of day.** A daily commit with a clear message is your safety net and your status log.
- **Keep a lab notebook.** A plain text file: what you ran, what happened, the numbers. Your weekly updates then write themselves.

When you get stuck (the 30-minute rule)

- **1. Read the actual error message** — not the first line, the whole thing. It usually says what's wrong.
- **2. Check the official doc** linked for that day — not a random blog.
- **3. Confirm versions match** — Jetson is very version-sensitive; a command for the wrong JetPack will fail.
- **4. Reproduce the smallest failing case** — strip it down until you find the one line that breaks.
- **5. After ~30–45 minutes stuck, ask your lead** — with what you tried and the exact error. That's good practice, not weakness.

Two things that will save you days

Environment/hardware issues are not learning exercises. If `nvidia-smi` fails or drivers are broken, that's a same-hour lead question, not a solo debugging marathon.

Commands in this handbook are illustrative. Jetson, JetPack, and library versions drift. Always verify exact syntax against the linked docs for **your** installed version.

6. The 20-day plan

Each day gives you: **why it matters**, **what to read first**, the **concepts** behind it, **what to do** (with example commands), the **pitfalls** to avoid, and a **done-when** line. Treat the done-when as your checklist — if you can produce it, move on. The day estimates assume a clean run; real days slip, which is exactly why Day 10 is a re-planning checkpoint.

WEEK 1

Foundations & getting models running on the Orin

Goal: each model runs on-device with honest baseline numbers.

DAY

1

Orientation & Environment Setup

Get a working machine and a clear picture of the goal

Why today matters. Before you touch a single model, you need a computer that works and a clear idea of what you're building. Most lost time in Week 1 isn't AI — it's environment problems. Nail the foundation now and the rest of the month is smoother.

READ FIRST

- Re-read the Project G brief. Focus on **what changed**: you're now using pre-trained models you mostly don't train, instead of training one from scratch.
- [Open-vocabulary & foundation models overview](#) — just absorb the shift from “train on my classes” to “prompt a pre-trained model.”
- [Foundation models on Jetson — NVIDIA](#) — these models are built to run on your exact device.

Concept: what is the Jetson Orin?

It's a small computer with a built-in NVIDIA GPU, made to run AI “at the edge” — in the field, with no internet. It runs Ubuntu Linux. **JetPack** is NVIDIA's software bundle that installs the GPU drivers, **CUDA** (the layer that lets code use the GPU), and core libraries. If the command `nvidia-smi` shows your GPU, the foundation is working.

THEN DO

1. Open a terminal on the Orin and run the GPU check (below). You should see a table listing the GPU and its memory.
2. Check the toolchain: CUDA compiler, Python, and that PyTorch can see the GPU (commands below). You want `True` from the torch check.
3. Confirm you can see camera data — view the live ZED feed or play a recorded bag file. You only need frames to arrive.
4. Create your Git repo (`git init`), commit a README stub, and start the habit of committing at the end of every day.
5. Write down and ask your lead the day-one questions (Section 7) — their answers reshape your whole plan.

Environment sanity checks (run each line)

```
nvidia-smi # shows the GPU + memory
```



```
nvcc --version          # CUDA compiler version
python3 --version
python3 -c "import torch; print(torch.cuda.is_available())"  # want: True
```

WATCH OUT FOR

- Jetson uses ARM64 chips. A plain `pip install torch` usually installs a broken build — use NVIDIA's prebuilt wheels or the Ultralytics Jetson docker image instead.
- If `nvidia-smi` fails, that's a hardware/driver issue for your lead — don't burn hours on it alone on day one.

DONE WHEN The GPU check works, `torch.cuda.is_available()` is True, you've seen camera frames, your repo exists, and your day-one questions are answered.

DAY
2

YOLO Running + First TensorRT Export

Get the fast detector live and compiled

Why today matters. YOLO is the easiest piece to win with first, because Ultralytics hides almost all the hard parts. Getting it running **and** TensorRT-compiled today gives you a working detector and your first hands-on taste of the optimize-for-speed workflow you'll repeat in Week 3.

READ FIRST

- [YOLO on NVIDIA Jetson — Ultralytics quick start](#) — focus on the install steps and the export command.
- [YOLO → TensorRT export](#) — focus on what `format=engine` does and the FP16 option.

Concept: what is TensorRT and an “engine”?

Normally your model runs through a general-purpose interpreter. **TensorRT** is NVIDIA's compiler that rebuilds the model into a hardware-specific **engine** file (`.engine`) tuned for your exact GPU — fusing operations and choosing the fastest code paths. Same model, same answers, often several times faster. On Jetson this step is **not optional** for real-time.

THEN DO

1. Install Ultralytics. Easiest path: pull the prebuilt Jetson docker image (the quick-start page gives the exact line for your JetPack version) — it already contains PyTorch, TorchVision and TensorRT for ARM64.
2. Sanity-check detection on a sample image (command below). You should get an annotated image with boxes.
3. Export to a TensorRT engine at FP16 (command below). This produces a `.engine` file. The first export is slow because it's compiling — let it finish.
4. Run inference from the engine and compare the per-image time printed before vs after. Record both numbers.

```
YOLO: predict, export, run engine
# 1. quick detection test
yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'

# 2. compile to a TensorRT engine (FP16)
yolo export model=yolo11n.pt format=engine half=True  # -> yolo11n.engine

# 3. run inference from the engine
yolo predict model=yolo11n.engine source='https://ultralytics.com/images/bus.jpg'
```

WATCH OUT FOR

- Engines are **device-specific** — one built on a laptop GPU will not work on the Orin. Always build on the Orin itself.
- Start with the smallest model variant (the `n` / nano size) before trying anything bigger — the Orin has limited memory.
- Exact flag names drift between versions; if a command errors, check the linked doc for your installed version.

DONE WHEN YOLO runs from a `.engine` file on the Orin and you've recorded FPS before vs after TensorRT.

DAY

3

NanoSAM Running

Get the segmentation engine producing masks

Why today matters. NanoSAM turns a box into a precise pixel mask — it's your segmentation engine. It's more hands-on than YOLO because you build the engine files yourself, so give it the full day and follow the repo's README exactly.

READ FIRST

- [NanoSAM — official NVIDIA repo README](#) — focus on the “build the engine” section and the short Python predictor example.
- [Knowledge distillation — NVIDIA intro](#) — just the idea: NanoSAM is a small model trained to copy a bigger one, which is why it's fast.

Concept: “promptable” segmentation

Unlike a classifier that outputs one label, SAM-style models take a **prompt** — a point (“this pixel”) or a box (“this region”) — and return a precise mask of the object there. NanoSAM doesn't know what the object is **called**; it just outlines it. That's exactly why you pair it with YOLO, which supplies the name and the box.

Concept: ONNX files → engines

NanoSAM ships as **ONNX** files (a portable model format). You convert each one into a TensorRT engine with `trtexec`, a command-line tool that comes with TensorRT. You'll build two: an **image encoder** and a **mask decoder**.

THEN DO

1. Clone the NanoSAM repo onto the Orin and read its README fully once before running anything.
2. Get the ONNX files the README points to (image encoder + mask decoder).
3. Build the engines with `trtexec`, using the README's exact flags (example below — but the README wins).
4. Run the Python predictor on a test image with a point prompt (snippet below). Save the mask overlaid on the image and confirm it outlines the object under your point. Note the latency.

Build an engine with `trtexec` (follow the README's exact flags)

```
/usr/src/tensorrt/bin/trtexec \  
--onnx=resnet18_image_encoder.onnx \  
--saveEngine=resnet18_image_encoder.engine --fp16
```

Run the predictor (Python)

```
from nanosam.utils.predictor import Predictor  
predictor = Predictor('data/resnet18_image_encoder.engine',
```

```

        'data/mobile_sam_mask_decoder.engine')
predictor.set_image(image)                # encodes the image once
mask, _, _ = predictor.predict(np.array([[x, y]]), np.array([1])) # 1 = foreground

```

WATCH OUT FOR

- The README notes the image encoder may need **FP32** (not FP16) to avoid errors — follow it rather than forcing FP16.
- Point labels matter: 1=foreground, 0=background, 2/3=box corners. The wrong label gives nonsense masks.
- If `trtexec` isn't found, it lives in the TensorRT install, often `/usr/src/tensorrt/bin/`.

DONE WHEN NanoSAM outputs a clean mask for a prompted object on the Orin, and you know its latency.

DAY 4

Understand the Combined Pipeline

See how the two models snap together — on paper first

Why today matters. You now have two working models. Before writing the glue that joins them, spend a day understanding exactly how they connect. Ten minutes of sketching saves hours of confused debugging tomorrow.

READ FIRST

- [YOLO + SAM combined framework \(paper\)](#) — focus on the detector-then-segmenter diagram; skip the heavy math.
- [YOLOE — open-vocabulary YOLO](#) — only if your front-end is text-prompted; how prompts become boxes.

Concept: how the two models talk

YOLO outputs, per object: a **class name**, a **confidence**, and a **box** (four numbers — `x1,y1,x2,y2` in pixels). NanoSAM wants a **prompt**. The “glue” is simply reshaping each YOLO box into NanoSAM's box-prompt format (corner points with labels 2 and 3). That's the whole interface between them.

THEN DO

1. On paper, draw the flow: image → YOLO → list of (name, box) → for each box → NanoSAM → mask → collection of (name, mask).
2. Inspect a real YOLO output in Python: print `results[0].boxes` and look at `.xyxy` (corners), `.cls` (class), `.conf` (confidence). Learn the exact shapes.
3. Read NanoSAM's `predict` signature and note exactly what a box prompt should look like (points array + labels array).
4. Write a throwaway script: run YOLO on one image, take the first box, convert it to NanoSAM's format, and print both — don't run NanoSAM yet, just prove you can produce the right input shape.

WATCH OUT FOR

- Coordinate conventions trip everyone: confirm both use (x, y) in pixels from the same origin (top-left). A flipped axis puts masks in the wrong place.
- YOLO can return many boxes or none. Decide now how you'll handle “no detections” and “dozens of detections.”

DONE WHEN You can explain (and have sketched) the full data flow, and your throwaway script prints a YOLO box correctly reshaped into a NanoSAM prompt.

Why today matters. You can't claim "real-time" later without knowing where you started. Today you measure each model's speed properly and learn the tools that tell you whether the Orin is actually keeping up.

READ FIRST

- [Jetson Orin overview](#) — focus on power modes and that the device throttles when hot.
- [Benchmarking on Jetson \(DeepStream guide\)](#) — focus on how FPS and latency are measured.

Concept: latency vs throughput vs "real-time"

Latency = how long one frame takes (ms). **Throughput / FPS** = frames per second. **Real-time** = throughput keeps up with the camera. Your target is $\geq 10 \text{ Hz} = \leq 100 \text{ ms per frame}$, measured **end-to-end** (the whole pipeline), not just one model.

Concept: power mode, clocks, tegrastats

`nvpmodel -m 0` sets max power mode (all cores on). `jetson_clocks` locks clocks to maximum. `tegrastats` prints live CPU/GPU/memory/temperature. Without max power on, the Orin runs slow to save energy and your benchmarks lie.

THEN DO

1. Enable max performance (commands below).
2. Run `tegrastats` in one terminal; run each model in another. Watch GPU%, RAM, and temperature.
3. Time each model alone (YOLO, NanoSAM) over ~100 frames and record average latency in a small table.
4. Note whether memory is tight (the Orin Nano shares ~8 GB between CPU and GPU) and whether temperature climbs.
5. Write your Week 1 half-page status update: what works, the numbers, what's next.

Turn on max performance, then watch the device

```
sudo nvpmodel -m 0      # max power mode
sudo jetson_clocks      # lock clocks high
tegrastats              # live CPU/GPU/RAM/temp (Ctrl-C to stop)
```

WATCH OUT FOR

- Always benchmark with max power mode **on**, or your numbers are meaningless.
- The first few frames are always slower (warm-up) — discard them before averaging.

DONE WHEN You have a baseline latency/resource table and your Week 1 status update is written.

WEEK 2

Wire the pipeline together + gather data

Goal: a working (not-yet-fast) end-to-end pipeline on real frames.

DAY

6

Connect YOLO → NanoSAM (single frame)

First real end-to-end detect-then-segment

Why today matters. Today the pipeline becomes real: one image goes in, named masks come out. Get it **correct on a single frame** before worrying about many frames or speed. This script is the backbone you'll improve all month.

READ FIRST

- Re-read the predictor sections of both repos — you'll be living in those two APIs today, especially their input/output shapes.

Principle: make it work, then make it fast

Do **not** optimize today. A slow-but-correct pipeline is the milestone. All speed work is Week 3. Chasing speed before correctness is the single most common way beginners lose a week.

THEN DO

1. Write a script: load image → YOLO → for each box, convert to a prompt → NanoSAM → mask.
2. Collect results as a list of (class name, confidence, mask).
3. Overlay all masks on the image, colour-coded by class, with the class label drawn on.
4. Save the annotated image and open it. Check the masks line up with real objects.
5. Commit this script — it's the spine of the whole project.

WATCH OUT FOR

- Encode the image **once** per frame, then reuse it for every box. `set_image()` does the encode; `predict()` reuses it. Re-encoding per box is slow and a classic mistake.
- Handle the empty case (no boxes) without crashing.

DONE WHEN One image produces correct, labelled masks, saved and eyeballed.

DAY

7

Data Collection Planning + Capture

Gather footage that reflects real conditions

Why today matters. Your models are pre-trained, but you still need real footage from the device — to test on, and possibly to evaluate or fine-tune. Variety of conditions matters far more than raw count.

READ FIRST

- The data-collection notes in the Project G brief. Focus on the principle: **diversity of conditions beats quantity**.

Concept: why variety matters

A model that only ever sees noon sunshine fails at dawn. Capture the conditions the device will actually face: different light, weather, distances, and backgrounds. Ten varied scenes are worth more than one scene filmed for ten minutes.

THEN DO

1. List the conditions that matter for your use case: times of day, weather, scene types, distances.
2. Capture footage with the SCANNER-01 across those conditions.
3. Save recordings with descriptive names, e.g. 2026-06-23_0830_overcast_field-edge.
4. Note for each clip: when, where, and what's in it.

WATCH OUT FOR

- Don't capture 500 near-identical frames of one scene.
- Back up the raw recordings before you start processing them.

DONE WHEN An organised, condition-labelled footage set exists.

DAY

8

Evaluation Set + (optional) Labelling

Decide zero-shot vs fine-tune, and build a fair test set

Why today matters. To know whether the pipeline is any good, you need **ground truth** to compare against. Today you decide whether you're labelling at all, and you build a test set that won't lie to you.

READ FIRST

- [CVAT getting started](#) — only if you're labelling.
- The idea of splitting data **by recording session, not by frame** (explained below).

Concept: you may not need to label much

Foundation models work **zero-shot** — out of the box, no training. If your lead wants zero-shot, you only label a small **evaluation** set to score quality, not a big training set. If they want fine-tuning, you need more labels. This is why the day-one question about fine-tuning matters so much.

Concept: data leakage

If frames from the same recording end up in both your test and training sets, your scores are inflated — the model has effectively already seen the test. Always split by **whole recording session**, never by individual frame.

THEN DO

1. Confirm with your lead: zero-shot (small eval set only) or fine-tune (larger labelled set)?
2. Pick a held-out evaluation set — recordings you will NOT tune on, and won't peek at.
3. If labelling: set up CVAT, write a one-page guideline, and label consistently.
4. Define your class set / text prompts — start with 5–8 clear categories.

WATCH OUT FOR

- Ambiguous labels ruin evaluation — write down edge-case rules (“is a fallen branch canopy or ground?”).
- Keep the eval set locked — don't look at it while tuning.

DONE WHEN A fair evaluation set and an agreed class/prompt list exist.

DAY

9

Pipeline Robustness on Real Frames

Find where it breaks before it has to be fast

Why today matters. Demo images are easy; real footage is messy. Today you run the pipeline across your captured frames to find where it breaks, so you can decide what's worth fixing.

READ FIRST

- Skim NanoSAM's notes on multiple prompts / batching — focus on feeding several boxes per frame efficiently.

THEN DO

1. Run your Day 6 pipeline over a whole folder of real captured frames.
2. Save the annotated outputs and scroll through them.
3. Catalogue failure modes: missed objects, wrong masks, tiny objects, overlaps, slow frames.
4. Fix crashes first (empty detections, odd image sizes); quality issues second.

WATCH OUT FOR

- Tiny or far-away objects are where SAM-style models struggle — note it, don't over-engineer a fix yet.
- Overlapping boxes can produce overlapping masks — decide a simple rule (highest confidence wins).

DONE WHEN The pipeline runs over many real frames without crashing, and you have a written failure-mode list.

DAY
10

Mid-Project Checkpoint

Demo what works, get feedback, re-plan honestly

Why today matters. Halfway. Time to show what works, gather feedback, and re-plan the back half. Plans always need adjusting by now — that's expected, not failure.

READ FIRST

- Your own Week 1–2 notes and yesterday's failure-mode list.

THEN DO

1. Run the full pipeline on a short live walk and record the result.
2. Demo to your lead — show the failure-mode list too; honesty builds trust and gets better advice.
3. Write your Week 2 status update.
4. Re-plan Weeks 3–4 based on where you actually are.

WATCH OUT FOR

- Resist hiding what doesn't work — the failures are the most useful thing to discuss.
- If you're behind, cut scope **now** (drop a stretch goal) rather than at the very end.

DONE WHEN A working (not-yet-fast) pipeline is demoed on real data, the Week 2 update is done, and Weeks 3–4 are re-planned.

WEEK 3

Optimise for real-time + ship a ROS 2 node

Goal: the pipeline runs live, in budget, as a deployable node.

DAY

11

Precision: FP16 Everywhere

The first big, nearly-free speedup

Why today matters. This is the first real-time lever after model choice. FP16 gives you roughly double the speed for almost no accuracy loss — start here before the trickier INT8.

READ FIRST

- [Quantization overview — PyTorch docs](#) — focus on FP32 vs FP16 vs INT8.
- [TensorRT intro — NVIDIA](#) — what compiling buys you.

Concept: precision in plain terms

Models store numbers. **FP32** = 32-bit (precise, slow, big). **FP16** = 16-bit (half the data, faster, tiny accuracy loss). **INT8** = 8-bit integers (fastest, smallest, needs care). Lower precision = less data to move = faster on the Orin's tensor cores.

THEN DO

1. Make sure both models' engines are built with FP16 (`--fp16 / half=True`).
2. Re-run your benchmark and compare against the Week 1 baseline.
3. Find the bottleneck: time each stage (image encode, detect, segment-per-box). Which one eats the budget?

WATCH OUT FOR

- The image encoder may need to stay FP32 (NanoSAM's README warns about this) — don't force FP16 if it breaks masks.
- With many boxes, the bottleneck is usually segmentation-per-box — that finding sets up Day 13.

DONE WHEN An FP16 pipeline with new latency numbers and a clearly named bottleneck.

DAY

12

INT8 Quantization + Calibration

The biggest remaining speedup — done carefully

Why today matters. INT8 is the largest speedup left, but it can hurt accuracy, so it's done carefully with calibration. This is the most technical day of the project — go slowly and check accuracy at the end.

READ FIRST

- [INT8 export & calibration — Ultralytics](#) — focus on calibrating **on the Orin**.
- [TensorRT quick start \(engine build\)](#)

Concept: what calibration is

INT8 squeezes the model's numbers into a tiny 8-bit range. To do that without wrecking accuracy, TensorRT runs a few hundred representative images through the model and measures the range of values it sees, so it can map them sensibly. That measuring step is **calibration**; the images are the **calibration set**.

Concept: calibrate on the device

The mapping is hardware-specific. Calibrate on the **same Orin** you'll deploy on, or the results drift and accuracy suffers.

THEN DO

1. Assemble a small, representative calibration set (a few hundred frames from your captures).
2. Export INT8 engine(s) **on the Orin**, with calibration (command below for YOLO).
3. Compare INT8 vs FP16: measure the speed gain AND the accuracy drop.
4. Decide per model whether INT8 is worth it — it's fine to keep one model FP16 and one INT8.

YOLO INT8 export with calibration (run on the Orin)

```
yolo export model=best.pt format=engine int8=True data=your_data.yaml
# 'data' points to a yaml that lists representative calibration images
```

WATCH OUT FOR

- A big accuracy drop usually means a poor calibration set — use more representative images.
- Calibration takes minutes and can look like it's hanging — it isn't, let it run.

DONE WHEN INT8 engine(s) built, plus a speed-vs-accuracy table that justifies your choice.

DAY

13

Resolution & Pipeline-Shape Tuning

The structural lever to get inside budget

Why today matters. If precision alone didn't reach 10 Hz, the **shape** of the pipeline is the next lever — and for a detector-plus-segmenter, it's often the biggest one.

READ FIRST

- Re-skim the Jetson benchmarking guide — focus on the resolution-vs-speed trade-off.

Concept: resolution scales work quadratically

Halving an image's side length quarters the pixels, and roughly quarters the work. Going from 1280 to 640 is a **4×** cut. Find the lowest resolution that still detects what you need.

Concept: segment less

You don't segment the whole frame — only the boxes YOLO found. Fewer/smaller boxes = less NanoSAM work. With many objects, consider segmenting only the top-N by confidence, or detecting every frame but segmenting every **other** frame.

THEN DO

1. Sweep input resolution; record speed and accuracy at each setting.
2. Confirm you segment only detected boxes, never the full image.
3. If still short: cap the number of boxes segmented per frame, or split detection/segmentation rates.
4. Re-benchmark end-to-end against the 100 ms budget.

WATCH OUT FOR

- Too-low resolution makes YOLO miss small objects — there's a floor; find it.
- Measure end-to-end, including the glue code, not just the two models.

DONE WHEN The pipeline runs inside (or close to) the 10 Hz / 100 ms budget.

DAY

Wrap as a ROS 2 Node

14

Turn the script into a deployable component

Why today matters. A standalone script isn't deployable — the rest of the SCANNER-01 system talks over ROS 2. Today you turn your pipeline into a proper component that subscribes to the camera and publishes results.

READ FIRST

- [ROS 2 publisher/subscriber \(Python\)](#) — focus on node, subscriber, publisher, callback.
- [ROS 2 for Beginners \(nodes & topics\)](#) — refresher if needed.

Concept: nodes, topics, publish/subscribe

A ROS 2 system is many small programs (**nodes**) that talk by sending messages to named channels (**topics**). Your node **subscribes** to the camera topic, runs the pipeline in a **callback** each time a frame arrives, and **publishes** the labelled result to a new topic that other projects can consume.

THEN DO

1. Create a ROS 2 Python package and a node file.
2. Subscribe to the camera image topic; in the callback, convert the ROS image to a numpy array (via `cv_bridge`) and run your pipeline.
3. Publish the labelled masks / labelled cloud on a new topic.
4. Test it: run the node and `ros2 topic echo` your output topic to confirm messages flow.

Minimal node shape (pseudocode)

```
class SegNode(Node):
    def __init__(self):
        super().__init__('seg_node')
        self.sub = self.create_subscription(Image, '/camera/image', self.cb, 10)
        self.pub = self.create_publisher(Image, '/seg/labels', 10)
    def cb(self, msg):
        frame = bridge.imgmsg_to_cv2(msg)    # ROS image -> numpy
        labels = run_pipeline(frame)         # your Day 6 pipeline
        self.pub.publish(bridge.cv2_to_imgmsg(labels))
```

WATCH OUT FOR

- If the callback is slower than the frame rate, frames queue up and you lag. Process latest-only or drop frames rather than fall behind.
- Image encodings (`rgb8` vs `bgr8`) via `cv_bridge` are a classic source of colour bugs.

DONE WHEN A ROS 2 node runs the pipeline live and publishes on a topic you can `echo`.

DAY 15

Integration + Live Test

Prove it runs inside the real system

Why today matters. Your node has to work inside the live system, competing for resources with other nodes — not just on its own. Today you plug into the stack and run a real walk-through.

READ FIRST

- Coordinate with Project B (fusion) and Project C (data store) on interfaces and which camera–LiDAR calibration version is current.

THEN DO

1. Run your node alongside the camera/fusion stack; confirm it receives real frames.
2. Do a live indoor walk; watch `tegrastats` for throttling and watch for frame drops.

3. Fix the worst real-time issues (queue buildup, throttling, memory pressure).
4. Write your Week 3 update.

WATCH OUT FOR

- Under sustained load the Orin heats up and throttles — sustained FPS is lower than a 10-second burst. Test for **minutes**, not seconds (this is exactly Project F's thermal concern).
- Other nodes compete for GPU and RAM — your solo benchmark was optimistic.

DONE WHEN The pipeline runs live in the stack, publishing in real time; Week 3 update done.

WEEK 4

3D integration, evaluation & hand-off

Goal: a labelled point cloud, measured results, and a clean hand-off.

DAY

16

Lift 2D Masks onto 3D Points

The project's payoff — a labelled 3D map

Why today matters. This is the payoff: turning flat image masks into a labelled 3D map. It's the trickiest geometry in the project, so give it a full day and lean on Project B's calibration.

READ FIRST

- [Point cloud basics — MathWorks](#) — focus on what (x, y, z) means.
- Project B's camera–LiDAR calibration — get the exact extrinsics and confirm the version.

Concept: projection and extrinsics

The LiDAR gives 3D points; the camera gives 2D labelled pixels. **Extrinsic calibration** is the known transform between the LiDAR's and camera's positions. Using it, you compute — for each 3D point — which camera pixel it lands on, then copy that pixel's label onto the point. That's “lifting” 2D labels into 3D.

THEN DO

1. Get the calibration from Project B: camera intrinsics + LiDAR-to-camera extrinsics.
2. For each LiDAR point: transform into the camera frame, project to a pixel, look up that pixel's mask label.
3. Build a labelled point cloud and visualise it in RViz or Foxglove, coloured by label.
4. Spot-check: do the 3D labels match reality?

WATCH OUT FOR

- Points behind the camera or outside the image must be **skipped**, not projected — they produce garbage pixels.
- A wrong or stale calibration version puts every label in the wrong place — confirm the version with Project B.

DONE WHEN A labelled point-cloud topic exists and looks correct in RViz/Foxglove.

DAY

17

Multi-Camera Conflicts + Project C

Resolve disagreements and persist labels

Why today matters. With three cameras, one 3D point can be seen by several — and they may disagree. Labels also need to be saved with recordings. Today you resolve conflicts and persist results via Project C.

READ FIRST

- Project C's plugin / recording interface — how labels get stored alongside recordings.

Concept: conflict resolution

If two cameras label the same point differently, you need a rule: majority **vote**, **highest-confidence** camera wins, or **weight by confidence**. Pick one, document it, and log when conflicts happen — don't resolve them silently.

THEN DO

1. Detect points seen by more than one camera.
2. Apply your chosen conflict rule.
3. Persist labels alongside recordings via Project C's data store.

WATCH OUT FOR

- Coordinate the storage format with Project C so they can actually read what you write.
- Don't drop conflicts silently — your rule should be explicit and logged.

DONE WHEN Conflict-resolved labels are persisted with recordings.

DAY
18

Final Evaluation

Numbers, not vibes — quality and speed

Why today matters. Now you prove it with numbers: how good are the labels, and does it really run in real time? And you're honest about what it can't do.

READ FIRST

- A refresher on how **mIoU** is computed from a confusion matrix, so you can interpret per-class results.

Concept: mIoU

For each class, Intersection-over-Union = the overlap between predicted and true mask ÷ their union. **mIoU** averages this across classes. 1.0 is perfect; the brief's bar is **≥0.5** for your chosen classes. Classes with few pixels can swing the average wildly — always report per-class too.

THEN DO

1. Run quantitative evaluation on the locked held-out set: mIoU per class.
2. Measure the final inference rate on the Orin — this is the real-time claim, in numbers.
3. Write an honest limitations section: which classes/conditions fail and why.

WATCH OUT FOR

- Evaluate on the held-out set **only** — never on data you tuned on.
- Report the average **and** the per-class breakdown; averages hide weak classes.

DONE WHEN An evaluation report covering quality, speed, and honest limitations.

DAY
19

Documentation & Reproducibility

Make it runnable by a stranger

Why today matters. Your work is only valuable if someone else can run it. A clear README is the difference between your project living on and dying the day you leave.

READ FIRST

- Your own repo — read it as if you'd never seen it, and note every place a newcomer would get stuck.

THEN DO

1. Write a README that takes someone from a fresh Orin to a running demo, step by step.

2. Document the engine-build commands, calibration steps, launch commands, and where files live.
3. Tidy the repo; make sure engines/checkpoints are findable or rebuildable.
4. Have someone (or yourself, in a clean shell) actually try the steps.

WATCH OUT FOR

- “Works on my machine” assumptions are the enemy — write down every environment detail and version.
- Big engine files may not belong in Git — document where to get or how to rebuild them instead.

DONE WHEN A stranger could reproduce your demo from the README alone.

DAY
20

Presentation & Hand-Off

Land it well and pass it on

Why today matters. Land the month well. A clear 15-minute story and a clean hand-off make your work count and help whoever continues it.

READ FIRST

- Re-read your weekly status updates to reconstruct the project's arc and the decisions you made along the way.

THEN DO

1. Build the 15-minute walkthrough: what was built, what failed, what should come next.
2. Record a demo video as a backup — live demos fail at the worst moment.
3. Hand off the repo and notes to the next intern or the lead, and walk them through it.

WATCH OUT FOR

- Show the failures and trade-offs, not just the highlight reel — it's more credible and far more useful.
- Leave a short “if I had another month” list — it's gold for whoever continues.

DONE WHEN Presentation delivered, demo video saved, and a clean hand-off complete.

7. Day-one questions for your lead

These answers decide how much of this handbook is central vs background. Ask them on Day 1:

1. **Front-end:** open-vocabulary YOLO (text prompts) or a fixed-class detector? Sets whether labelling matters at all.
2. **Is NanoSAM the segmentation target**, or a different SAM variant?
3. **Do we fine-tune anything**, or run the models zero-shot and use our data only for evaluation?
4. **What class set / prompts** matter for the use case? (Propose 5–8 to start.)
5. **What frame rate** is “good enough” — is 10 Hz the real target or just a floor?

If you only remember one thing

It's one pipeline: **open-vocab YOLO detects and names** → **NanoSAM makes precise masks** → **masks get lifted onto the 3D map** — all compiled with TensorRT and run in reduced precision so it keeps up with the camera. Everything else hangs off that spine.

8. Key resources

Orientation — read first

- [Open-vocabulary & foundation models overview \(2026\)](#)
- [Foundation models on Jetson — NVIDIA](#)

The models

- [NanoSAM — official NVIDIA repo](#)
- [MobileSAM — official repo](#)
- [Knowledge distillation — NVIDIA Jetson intro](#)
- [YOLO on NVIDIA Jetson — Ultralytics](#)
- [YOLOE — open-vocabulary YOLO](#)
- [YOLO + SAM combined framework \(paper\)](#)

Optimisation — getting to real-time

- [TensorRT intro — NVIDIA blog](#)
- [TensorRT quick start — official docs](#)
- [YOLO → TensorRT export — Ultralytics](#)
- [Quantization overview — PyTorch docs](#)
- [YOLO real-time on Jetson w/ DeepStream](#)

Deployment glue

- [ROS 2 publisher/subscriber \(Python\)](#)
- [Point cloud basics — MathWorks](#)
- [CVAT getting started](#)